Pearl INT 白皮书

Pearl Research Labs

中文译本

摘要

本文档说明 Pearl 网络的设计。Pearl 是一种工作量证明有用工作（Proof-of-Useful-Work，PoUW）第一层协议；挖矿工作原生地来自 AI 计算。协议的核心是一种高效实现的 GPU 基础运算——矩阵乘法——使 GPU 能够在执行 AI 训练和推理工作负载的同时，以副产物的形式完成工作量证明，即“二合一”。因此，Pearl 将能源、数据和货币结合为一种不可分割的数字资产。本文概述协议设计、关键实现选择及系统的若干经济方面。

目录

1. 引言

1.1 计划中的升级

2. 区块链概述

3. 有用工作证明

3.1 高层说明

3.1.1 低秩噪声范式

3.1.2 带执行轨迹的矩阵乘法

3.1.3 工作量证明与成功挖矿

3.1.4 哈希与承诺

3.1.5 隐私与零知识工作量证明

4. 实现细节

4.1 挖矿配置

4.2 MatMul 框架

4.3 承诺哈希

4.4 噪声矩阵生成

4.5 分块 MatMul 算法

4.6 区块开启证明

4.7 ZK-SNARK 区块开启证明

4.8 支持的 PoW 参数

5. 技术规格

5.1 区块结构
5.2 地址
5.3 虚拟机与脚本扩展
5.4 手续费、出块时间与发行
6. 经济概述
6.1 工作成本的分配
6.2 铸币、安全性与区块奖励
6.3 瞬时均衡
6.4 代币价格升值
7. Pearl 的发布

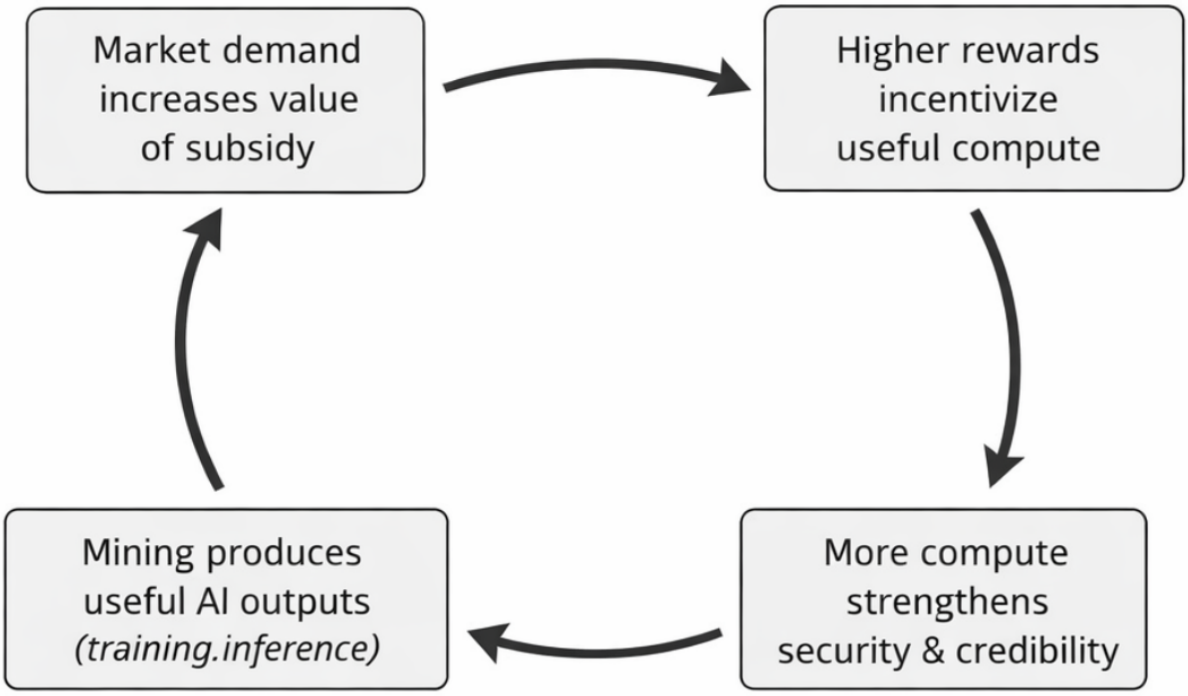
1. 引言

比特币最重要的概念贡献之一，是把电力转化为货币：稀缺且可验证的能源能够转化为数字稀缺性与可信中立性。与此同时，比特币挖矿依赖随机哈希这一人工、专用的计算任务，今天主要适用于 ASIC。AI 则正在迅速成为重要的用电来源；在大语言模型时代，AI 进步的核心限制越来越不是模型、算法或 GPU 本身，而是训练和推理所需能源的生产与可获得性。

Pearl 的中心命题是：以现代 AI 的原生操作矩阵乘法（MatMul）替代比特币的人工哈希挖矿，将通用 GPU 计算转化为货币，并直接利用 AI 增长来保护网络。区块奖励与可验证的矩阵乘法数量直接对应；启用 Pearl 后，花在 AI 上的每一度电可以同时赚取挖矿奖励并保护一个去中心化网络。

传统 PoW 只把安全性货币化。Pearl 同时将安全性和效用货币化：矿工能够用训练或推理工作负载挖矿，因此从同一 GPU 周期获得第二条收入来源。需求抬升挖矿补贴的价值；更高奖励带来更多有用算力；算力又提高共识安全性及货币可信度；挖矿输出的训练或推理结果则继续积累真实经济价值。

图 1：市场正循环



在工程上，Pearl 将新的 MatMul 挖矿核集成进现有 AI 框架和运行时。该插件式实现只在原始调用上增加极少操作；分块、核级、并行的方式也可利用空闲碎片、流水线停顿和微批次，使原本浪费的余量成为可参与出块的工作。长期而言，该网络还可支持算力市场、算力合约的链上结算，以及不同硬件和精度任务之间的价值结算。

1.1 计划中的升级

当前网络的核心是精确的整数（INT）矩阵乘法，可作为适合该设置（例如部分推理）的应用替代实现。但现代 AI 许多工作负载原生使用低精度浮点格式：训练常用 BF16，推理常用 FP8 与 FP4。低精度带来吞吐量与 I/O 的优势，却也使 PoUW 面临非均匀动态范围和数值稳定性问题。

Pearl 正在开发后续 PoUW 方案：它不再保持精确矩阵乘法，而是计算高精度的近似 MatMul，类似量化乘法。该方案会把原生量化噪声作为 PoW 所需的扰动机制，并要求矿工承诺量化前的权重与激活矩阵。完整的形式规格与安全性、准确性分析将在充分测试后公开；升级意在接纳训练工作负载，并原生支持最先进的低精度推理。

2. 区块链概述

Pearl 是从比特币协议分叉而来的 PoUW 区块链。它保留比特币的安全与共识模型，同时以矩阵乘法变体替代基于哈希的工作量证明，并加入相应的协议改造。账本采用 UTXO（未花费交易输出）模型：交易消耗已有 UTXO 作为输入，创建新的 UTXO 作为输出；每个输入都由私钥签名授权。

节点通过类似 gossip 的 P2P 网络传播交易和区块。账本是一条线性区块序列；每个区块含交易批、时间戳、前一区块哈希和满足 PoW 条件的证明。难度目标随网络竞争程度调整，以保持预期出块时间。节点采用累计工作量最大的链作为有效链，临时分叉会随着链延长而收敛。

为防双花和保证排序，只有最长链中的已确认交易才被视为最终确定。矿工得到新铸造的 Pearl 和交易费；若要破坏网络，攻击者必须控制网络中绝大部分解题能力，因而成本在经济和物理层面都极高。全节点维护可花费 UTXO 集，对每笔交易检查引用存在、未花费、签名有效且不存在双花。

3. 有用工作证明

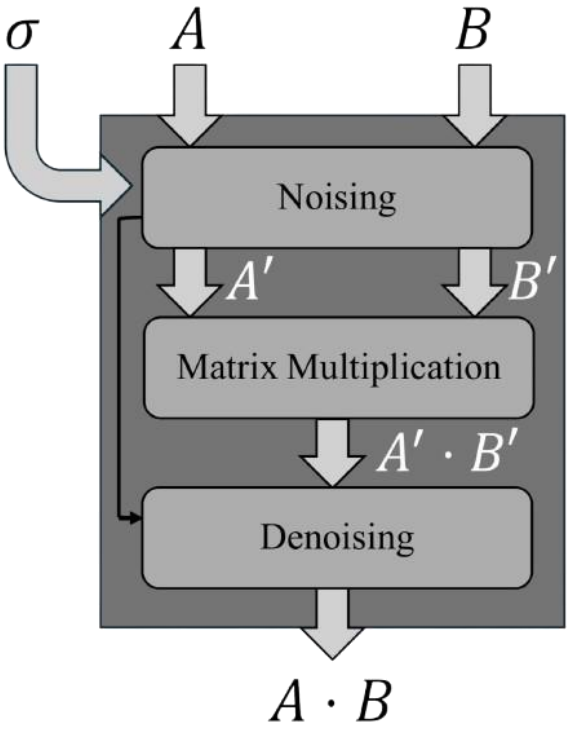
PoUW 的目标有二：既满足安全区块链对 PoW 的必要属性，又完成有用计算，即任意两矩阵的乘积。该协议将证明生成与矩阵乘法合并，使总运行时间尽可能接近矩阵乘法本身。证明不仅必须认证输出 A·B 正确，还必须认证规定的乘法算法确实被执行，以防矿工对零矩阵等退化输入投机取巧。

矩阵乘法是机器学习最基本的操作。虽然理论上存在更快的渐近算法，实际高性能硬件普遍使用内存优化的朴素算法变体，因为它们更符合现代内存层级和向量执行模型。其运行时间主要由矩阵维度决定，而不是元素值，因此适合作为公平的工作度量。

3.1 高层说明

协议为输入 A、B 加入精心构造、随后可移除的噪声矩阵 E、F。矿工计算扰动后的乘积 (A+E)·(B+F)，从执行轨迹中提取密码学证明；随后用快速后处理恢复原始 A·B。噪声既要令扰动乘法像随机矩阵乘法一样难，又必须允许原结果被高效、准确地恢复。

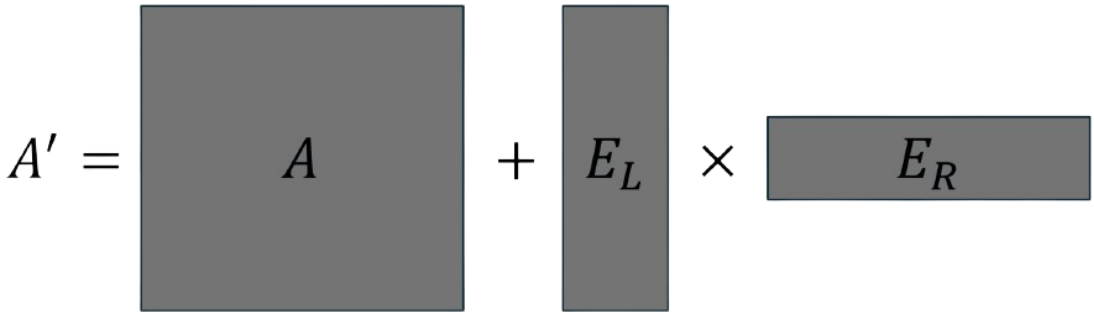
图 2：MatMul 框架



3.1.1 低秩噪声范式

E、F 是由输入 A、B 与链状态 σ 的密码学承诺确定性导出的秩 r 随机矩阵。低秩使校正项 E·B、A·F、E·F 可在 $O(n^2r)$ 标量乘法内完成；当 r 远小于 n 时，该成本相对于主乘法可忽略。

图 3：噪声生成



$$A \cdot B = (A + E)(B + F) - (E \cdot B + A \cdot F + E \cdot F)$$

这一恒等式同时带来风险：若矿工已有 A·B，可用低秩校正廉价模拟扰动乘积。为杜绝捷径，PoW 要求矿工不仅给出最终结果，还须给出其由有效矩阵乘法算法直接执行得到的可验证轨迹。

3.1.2 带执行轨迹的矩阵乘法

实际矩阵乘法通常是分块执行。将 A、B 分解为 t×t 块，计算 C=A·B 的每个输出块时，矿工必须给出每个块积及其部分和。若 k=n/t，则中间部分和可写为：

$$C_{ij,k}' = \Sigma_{l=0}^{k'} A_{il} \cdot B_{lj}$$

当 t≤r 时，全局低秩噪声在每个 t×t 局部块上呈现为边际上的满秩噪声。配合完整轨迹要求，这会迫使矿工逐块完成真实计算，而非仅构造最终输出。

3.1.3 工作量证明与成功挖矿

矿工为每个输出块 (i,j) 维护一个初始化为零的 512 位状态 M_{ij}。每轮累加后，把当前累加器 $C_{ij,k}$ 的所有元素异或压缩为 32 位值，并以旋转异或方式混入 M_{ij}。完整块在下列条件成立时开启，即成为一个有效 PoW：

$$\text{BLAKE3}(\text{seed}, M_{ij}) < 2^{256-b}$$

这里 b 是全局难度参数，哈希输出按 256 位无符号整数解释。成功证明包含 A、B 的承诺、挖矿配置和矩阵维度、开启块的索引，以及证明指定的带噪分块乘法确实产生满足阈值的累积哈希状态。

3.1.4 哈希与承诺

协议对 A、B 使用密码学承诺。承诺必须允许在给定矩阵根的情况下验证任意行/列条带确属该矩阵；Merkle 树可实现这一点。承诺还必须具有绑定性，使矿工无法找到两份不同矩阵却得到同一根，因而无法在噪声种子确定后再为其量身选择输入。

哈希函数还需从带噪块输出中抽取近似均匀的随机性，并相对矩阵乘法具有可忽略的开销。理论上 t×t 乘法约为 $\Theta(t^3)$，哈希为 $O(t^2)$；充分大的块使后者不成为瓶颈。噪声和中间哈希块均由承诺种子产生，实现 Fiat-Shamir 变换。

3.1.5 隐私与零知识 PoW

有用计算可能包含专有权重或用户敏感数据。承诺与由其导出的噪声允许验证者在不公开 A、B 内容的前提下验证协议，但获胜条带仍可能泄露部分内容。因此协议引入零知识证明，证明“存在与 A、B 承诺一

致的条带，其乘积得到累计摘要 M_{ij}，且 BLAKE3 哈希等于公开值 h”。验证者只公开检查 h 是否达到挖矿阈值。

关于 ASIC 抵抗性：矩阵乘法是 GPU、TPU 等现代 AI 加速器的原生语言，已受到整个半导体行业的持续优化。专为 Pearl 设计的 ASIC 会失去执行有用工作的能力，因此在经济上未必更有利。

4. 实现细节

4.1 挖矿配置

- 公共维度 k：A、B 的共同维度。
- 噪声秩 r：加入并移除的低秩噪声所用秩。
- 块形状 (t_m,t_n)：A 的行与 B 的列的周期性分块。
- 难度目标 b：每个 int8 乘加操作的对数接受概率；b 越大，接受的块越少。
- MatMul 累加类型：所用精确乘法算法的标识；初始值 0 表示输入在 [-64,64] 且以 int32 累加。

4.2 MatMul 框架

给定 A、B，协议首先以 A、B、配置 μ 和链状态 σ 运行承诺哈希，得到种子 sA、sB；再生成 $E=E^LE^R$ 与 $F=F^LF^R$，构造 $A'=A+E$、$B'=B+F$。随后按块运行 MatMul，并从开启块收集证明。由于因子的形状，最终可以快速去噪：

$$A\cdot B = A'\cdot B' - (A\cdot F^L)\cdot F^R - E^L\cdot(E^R\cdot B')$$

在 8 位整数协议中，A、B 被量化到 [-64,64]，E、F 到 [-63,63]，从而带噪矩阵仍能放入 int8 而不溢出。

算法 1：带噪 MatMul 框架

输入：矩阵 A、B，矿工配置 μ，状态 σ
输出：乘积 C=AB，以及零个或多个开启块 Blocks
1. (sA,sB) ← CommitmentHash(A,B,μ,σ)
2. (E^L,E^R) ← NoiseGeneration(m,k; key=sA)
3. (F^L,F^R) ← NoiseGeneration(n,k; key=sB)
4. $A' \leftarrow A+E^LE^R$; $B' \leftarrow B+F^LF^R$
5. C′, Blocks ← TiledMatMul(A′,B′)
6. $C \leftarrow C' - (AF^L)F^R - E^L(E^RB')$
7. 返回 C, Blocks

4.3 承诺哈希

噪声种子 sA、sB（亦称承诺哈希）依赖于 A、B 的 BLAKE3 哈希 HA、HB、挖矿配置 μ 和链状态 σ。HA 按行主序解析 A，HB 按列主序解析 B；这种不对称使开启证明只需揭示少量 A 行和 B 列，并允许在推理中预先处理通常已知的 B。

算法 2：计算承诺哈希

输入：张量 A、B，密钥 σ
输出：A、B 的承诺哈希
1. κ ← BLAKE3(σ || μ)
2. HA ← BLAKE3(Flatten(A), key=κ)
3. HB ← BLAKE3(Flatten(B^T), key=κ)

4. sB ← BLAKE3(κ || HB)
5. sA ← BLAKE3(sB || HA)
6. 返回 (sA,sB)

4.4 噪声矩阵生成

以 sA、sB 为种子，生成 $E=E^LE^R$、$F=F^LF^R$。E^L 的每一项由 keyed BLAKE3 伪随机生成器在有符号 6 位整数中均匀抽取；E^R 为按列的选择矩阵，每列在随机不同的两个位置各有一个 1 与 -1。F 的因子用 sB 同样生成，但分布方向与 E 对换。

- E、F 的秩为 r，可高效剥离。
- E、F 的每个条目有较高熵，抑制无用的加速。
- 即使 A=B=0，利用 E、F 的因子重排来计算全部块也不会出现明显加速，前提是 $k \le O(r^2)$。

$$EF=(E^LE^R)(F^LF^R)=E^L(E^RF^LF^R)=E^L(E^RF^L)F^R$$

算法 3：噪声生成

输入：维度 m,k，秩 r，密钥 s
输出：张量 E^L,E^R
1. $E^L \leftarrow \text{UniformMatrix}(s) \in [-32,31]^{m\times r}$
2. $E^R \leftarrow \text{ChoiceMatrix}(s) \in [-1,1]^{r\times k}$
3. 返回 E^L,E^R

4.5 分块 MatMul 算法

标准分块算法把 A′、B′ 分为 $t_m \times r$ 与 $r \times t_n$ 的块。每个输出块 C_{ij} 的累加器初始为零，并依次加上相应输入块的乘积。哈希状态只在完整秩块上更新：把当前累加器各元素做 int32 异或，写入由 16 个 int32 构成的 512 位状态 M。

$$C^{(l)}_{ij} \leftarrow C^{(l-1)}_{ij} + A'^{l}_{i} \cdot B'^{l}_{j}$$

$$\text{BLAKE3}(M^{(\lfloor k/r \rfloor - 1)}) \le 2^{256-b} \cdot r \cdot t_m \cdot t_n$$

算法 4：带累积哈希检查的分块 MatMul（摘要）

输入：A′, B′；输出：C′=A′·B′ 与开启块列表
1. Blocks ← [ ]；初始化 C′ 为 m×n 零矩阵
2. 对每个输出块 (i,j)：令 Cblk←0，$M \leftarrow 0^{16}$
3. 对 ℓ=0 到 ⌊k/r⌋：累加对应 A′、B′ 子块
4. 若块为完整 $t_m \times t_n$ 且深度为 r：$X \leftarrow \oplus$ Cblk；$M[\ell \bmod 16] \leftarrow (M[\ell \bmod 16] \ll 13) \oplus X$
5. 写入 C′；若 BLAKE3(M,key=sA) 达阈值，则将 Cblk 加入 Blocks
6. 返回 (C′, Blocks)

4.6 区块开启证明

开启证明向验证者提供重建候选块、检查内外哈希谓词所需的一切信息。它包括 A、B 的 BLAKE3 矩阵承诺；对应行和列的叶数据、叶索引及 Merkle 路径；块的行列索引和深度；以及矩阵形状 (m,n,k)、秩 r 与块形状 (t_m,t_n)。验证者先验证 Merkle 证明，再从承诺、链状态与配置导出噪声种子，重建候选块，并测试难度 b1 的内层哈希和难度 b2 的外层哈希。

4.7 ZK-SNARK 区块开启证明

上述基于 Merkle 片段的证明易于理解，但面临大小与隐私问题。对于两个 16384×16384 矩阵，块大小 16 时证明接近 0.5 MB，块大小 64 时接近 2 MB；将它们放入区块头会伤害扩展性，而且行列条带可能含个人或专有信息。

零知识简洁非交互知识论证（zkSNARK）可同时解决两项问题：证明者可证明其私有输入与公开链状态经验证器运算后得到 M，其 BLAKE3 哈希为 h，而不泄露输入。证明可压缩到 KB 量级（取决于后端）。文中选择仅依赖哈希函数的 hash-based zkSNARK：其证明与验证快、无需可信设置，且安全性只依赖哈希困难性，具备后量子安全特征。

实现采用 Plonky2，并作三项优化：第一，直接以算术中间表示（AIR）手工表达验证器电路，而非由高级语言编译，从而精细控制约束；第二，以双方约定的预处理列提供明文噪声，避免在零知识电路中证明昂贵的噪声导出；第三，以三层递归先快速生成证明、再强力压缩，最终证明小于 60 KB。zkSNARK 公开的内容包括 r、k、t_m、t_n、m、n、开启块位置 i、j、HA、HB，以及 BLAKE3(M,key=sA)。

4.8 支持的 PoW 参数

协议初始支持精确整数 MatMul：输入与带噪输入使用 int8，累加使用 int32。$m,n \le 2^{24}$；共同维度须满足 $16r \le k \le 4r^2$、$k \le 2^{16}$ 且 64 整除 k。秩 r 只能取 $\{2^5,2^6,\ldots,2^{10}\}$。行块大小 h 与列块大小 w 必须满足 $h \cdot w \ge 32$，以保证 M 有足够熵；验证器还限制 $k(h+w) \le 2^{22}$。

每个块形状 t_m、t_n以三维等差级数 (S1,S2,S3,L1,L2,L3) 表示：$T=\{\ell 1 S1+\ell 2 S2+\ell 3 S3 : 0 \le \ell i < Li\}$，其中 S1|S2|S3。块的形式为 t+T；任何部分落在 m、n 边界之外的块不得用于工作量证明。

5. 技术规格

5.1 区块结构

Pearl 将 PoUW 嵌入比特币分叉链。与比特币 80 字节区块头自行充当 PoW 证明不同，Pearl 的证明以 zkSNARK 编码可能很大的矩阵 A、B。因此去掉 nonce 字段，在区块头旁增加变长的 block certificate（区块证书）作为区块头有效性证明：certificate := version || bytes。证书与头部分离，使头格式稳定并允许未来升级；证书上限为 65 KB。

零知识证书是随机且非唯一的：同一有效见证可仅通过重采样证明者随机性生成多个不同、等效的证明，且不增加有用工作。因此 certificate 字节不能决定区块身份，否则矿工能事后任意改变区块 ID。区块身份仍为头部数据的双 SHA-256：hdr* := version || prev_hash || tx_root || time || nBits || pouw_meta，共 116 字节。pouw_meta 是对 zkSNARK 公共见证的 32 字节 SHA-256 承诺，把多个可能证书绑定到同一有用工作实例。

5.2 地址

比特币长期积累了 P2PKH、P2SH 和 SegWit 等多种地址格式；并存会造成碎片化、实现复杂度和更大的攻击面，且大多数旧格式直接依赖 ECDSA 或 Schnorr，面临量子威胁。Pearl 不支持旧地址类型，统一采用 Taproot 地址，以获得统一表示、效率、隐私与面向量子升级的路径。

为准备后量子安全，协议预留 OP_CHECKXMSSSIG（#222），功能类似 OP_CHECKSIG，但验证扩展 Merkle 签名方案（XMSS）。它用 SHAKE256，为 256 位消息提供约 256 位安全性；每个密钥对可签 32 次，签名大小为 2340 字节。该操作码当前未在节点共识中启用，待 Schnorr 被认为不安全时可激活。钱包在 Taproot 脚本路径中加入 XMSS 公钥，并沿 m/222'/<coin_type>'/account'/branch/index 推导密钥。另实现 BIP-360 的 P2MR：它取消易受量子攻击的 key-path 花费，所有花费都经脚本路径和 Merkle 证明，从而降低公钥暴露。

5.3 虚拟机与脚本扩展

Pearl 采用基于比特币、栈式且非图灵完备的虚拟机，并启用 OP_CAT。给定两个字节串 a、b（b 在栈顶），OP_CAT 弹出二者并压入 a||b。为防过度内存使用，输出长度上限为 520 字节，操作按 [lengthout/64] 计入 tapscript 动态预算，类似 OP_CHECKSIG 的固定成本。

5.4 手续费、出块时间与发行

费用由交易对区块空间的竞争形成。目标出块间隔设为 194 秒（3 分 14 秒），以在全球传播安全余量内改善最终性、链上响应与经济吞吐量。总供应量为 S=2,100,000,000 枚；最小单位为一枚的 10^{-8}，称为 grain。

发行采用平滑、多项式衰减曲线，以避免比特币式减半带来的突变激励和波动，并保留有限但较厚的长期安全发行尾部。四年对应 H=650,226 个预期区块，定义剩余供应比例及累计分配比例为：

$$R(t)=H/(t+H)；A(t)=1-R(t)=t/(t+H)$$

第 t 个区块的离散发行量是 S·A(t) 的差分，实际发行量按 grain 向下取整：

$$E^*(t)=S\cdot H\ /\ ((t+H)(t+H-1))$$

难度使用加权目标指数移动平均（WTEMA）。若 t 为当前区块与父区块的解题时间、T=194 秒、N·T 为约 7 天的特征衰减时间，则：

$$\text{target_new} = \text{target_old} + \text{target_old}\cdot(t-T)/(N\cdot T)$$

初始难度较低（nbits=0x1b00ffff）。区块时间戳必须比父区块晚至少 1 秒，且不得比 UTC 当前时间领先 5 分钟。由于 WTEMA 可能让相邻区块难度不同，累计工作近似相等的竞争链按先收到者打破平局；工作量差超过较小累计工作的四分之一时才恢复最重链规则。

6. 经济概述

本节给出一个入门级竞争均衡模型，用于理解 Pearl 这类 PoUW 网络的价格和挖矿水平。核心结论是：在自然假设下，PoUW 代币价格应跟随目标有用工作的增长；对 Pearl 而言，目标工作包括全球 AI 训练和推理。作为纯价值储存，代币价值仍依赖于未来会被足够多用户采用为支付手段的预期；若该前提不成立，代币价值和挖矿量均应为零。

6.1 工作成本的分配

可参与的世界挖矿算力由两类处理器组成：可把“有用工作”与挖矿结合的处理器，以及没有这类工作的处理器。记每 H100 GPU 小时的算力成本为 C。文中的示例把没有有用工作的处理器成本近似为当前市场计

算价格 C≈1.7 美元/H100 小时；有用工作处理器只承担额外开销，约为 τ·C，其中 τ≈0.1。真实成本分布可理解为两团直方峰：左边是有用工作的低额开销，右边是无用工作的完整成本。

6.2 铸币、安全性与区块奖励

设 e(t) 是时点 t 的代币发行速率（枚/小时），完全稀释供应量为 $S=\int_0^\infty e(t)dt$。每个时点的发行按矿工算力比例分配，总算力就是该时点网络安全性，因为攻击者须控制多数算力。以 Pearl 为例，$S=2.1\times10^9$，H=4 年=35,040 小时；发行曲线使 H 时仍有半数代币未分配，并使时点 T 的未发行量为 S·H/(H+T)。

6.3 瞬时均衡

瞬时均衡的条件是挖矿净计算成本等于所铸代币价值：

$$\tau \cdot G(t) \cdot D(t) = p(t) \cdot e(t)$$

其中 G(t) 为用于挖矿的 GPU 数，D(t) 为每 GPU 小时算力成本，p(t) 为每枚代币美元价格。若左侧大于右侧，矿工会退出；反之则会有更多算力进入。价格高于成本所暗示的水平可能有两种原因：所有有用工作已经被吸纳，或持有有用工作的公司因信任、认知、监管、行政成本或延迟而尚未接入。后者在启动日尤其可能发生；未来采用预期会抬升当前价格，并逐步吸引矿工。

文中把 $F(t)=p(t)e(t)/(\tau G(t)D(t))>1$ 称为未来预期乘数。示例假定有 100 万张有用工作 GPU、200 万张无用工作 GPU，启动日 e(0)=59,931 枚/小时：价格与挖矿量先线性相关，直到有用工作全部加入；之后若价格继续因预期上升，矿工获得更高净收益；再进一步时，即使没有有用工作的处理器也会参与。完整一般均衡模型还考虑 AI 价格内生变化：挖矿收入会迫使服务商给 AI 成本回扣，AI 价格降低又扩大需求和后续挖矿。

6.4 代币价格升值

该节的假设是：世界计算力增长时，Pearl 保持相同的世界计算力份额；并令效率参数 τ 不变。对 t1<t2，由均衡条件可得：

$$\rho p = p(t_2)/p(t_1) = \rho G \cdot \rho D / \rho e$$

其中 $\rho G=G(t_2)/G(t_1)$，$\rho D=D(t_2)/D(t_1)$，$\rho e=e(t_2)/e(t_1)$。发行曲线使 $\rho e=((H+t_2)/(H+t_1))^2$；例如第一年其倒数约为 $(5/4)^2\approx1.56$，第十年约为 $(15/14)^2\approx1.07$。因而该模型将代币价值视作对全球有用计算成本（略有额外发行衰减溢价）的“索引”。若计算力持续大致年翻倍、成本约每两年减半且系统保持瞬时均衡，文中预期代币价值可能略高于每两年翻倍；这依赖于上述假设，不构成价格保证。

7. Pearl 的发布

节点代码公开后，链即进入运行状态。团队同时提供 vLLM 插件，用新的量化机制以“二合一”方式重实现 DNN 层，并发布通过 Pearl 认证的 Meta Llama 3.3 70B 版本。下表比较原始 llama-3.3-70B-Instruct 与认证变体；测试均在 4×H200 GPU 上完成。TMAD 表示 Tera MAD（乘加）操作数，Useful MADs 为可用于挖矿的既有 MAD 总量。

表 1：原始 Meta LLaMA 70B 与 Pearl 认证“二合一”变体

模型	并行方式	MMLU	吞吐量（tok/s）	时间（秒）	有用 MAD（TMAD/s）
Meta LLaMA 70B	PP=4	0.8198	15,269.81	441.100	—

Meta LLaMA 70B	TP=4	0.8193	13,218	510	—
Meta LLaMA 70B	DP=2, TP=2	0.8197	13,162	512	—
Meta LLaMA 70B	DP=4	显存不足	bf16 模型约 140 GB 超出单 GPU 显存		
Pearl 认证版本	PP=4	0.8190	17,206.26	391.457	806
Pearl 认证版本	TP=4	0.8180	13,264.38	507.789	620
Pearl 认证版本	DP=4	0.8198	18,291.66	368.229	981

版权所有 © 2026 Pearl Research Labs。本文为上述官网 INT 白皮书的中文译本；变量、算法名称、密码学原语、数值与公式按原文保留。